

[On This Page](#) >

Troubleshooting Vars

Quick reference for diagnosing and fixing vars issues.

Common Issues

Generator Script Fails

Symptom: Error during `clan vars generate` or deployment

Possible causes and solutions:

1. Missing runtime inputs

```
1 # Wrong - missing required tool
2 runtimeInputs = [ ];
3 script = '
4     openssl rand -hex 32 > $out/secret # openssl not found!
5 '
6
7 # Correct
8 runtimeInputs = [ pkgs.openssl ];
```

2. Wrong output path

```
1 # Wrong - must use $out
2 script = '
3     echo "secret" > ./myfile
4 '
5
```

```

6 # Correct
7 script = ''
8   echo "secret" > $out/myfile
9 '';

```

3. Missing declared files

```

1 files."config" = { };
2 files."key" = { };
3 script = ''
4   # Wrong - only generates one file
5   echo "data" > $out/config
6 '';
7
8 # Correct - must generate all declared files
9 script = ''
10  echo "data" > $out/config
11  echo "key" > $out/key
12 '';

```

Cannot Access Generated Files

Symptom: "attribute 'value' missing" or file not found

Solutions:

1. Secret files don't have `.value`

```

1 # Wrong - secret files can't use .value
2 files."secret" = { secret = true; };
3 # ...
4 environment.etc."app.conf".text =
5   config.clan.core.vars.generators.app.files."secret".value;
6
7 # Correct - use .path for secrets
8 environment.etc."app.conf".source =
9   config.clan.core.vars.generators.app.files."secret".path;

```

2. Public files should use `.value`

```

1 # Better for non-secrets
2 files."cert.pem" = { secret = false; };

```

```
3 # ...
4 sslCertificate =
5     config.clan.core.vars.generators.ca.files."cert.pem".value;
```

Dependencies Not Available

Symptom: "No such file or directory" when accessing `$in/...`

Solution: Declare dependencies correctly

```
1 clan.core.vars.generators.child = {
2   # Wrong - missing dependency
3   script = ''
4   cat $in/parent/file > $out/newfile
5   '';
6
7   # Correct
8   dependencies = [ "parent" ];
9   script = ''
10   cat $in/parent/file > $out/newfile
11   '';
12 };
```

Permission Denied

Symptom: Service cannot read generated secret file

Solution: Set correct ownership and permissions

```
1 files."service.key" = {
2   secret = true;
3   owner = "myservice"; # Match service user
4   group = "myservice";
5   mode = "0400";       # Read-only for owner
6 };
```

Vars Not Regenerating

Symptom: Changes to generator script don't trigger regeneration

Solution: Use `--regenerate` flag

```
clan vars generate my-machine --generator my-generator --regenerate
```

Prompts Not Working

Symptom: Script fails with "No such file or directory" for prompts

Solution: Access prompts correctly

```
1 # Wrong
2 script = ''
3     echo $password > $out/file
4 '';
5
6 # Correct
7 prompts.password.type = "hidden";
8 script = ''
9     cat $prompts/password > $out/file
10 '';
```

Debugging Techniques

1. Check Generator Status

See what vars are set:

```
clan vars list my-machine
```

2. Inspect Generated Files

For shared vars:

```
ls -la vars/shared/my-generator/
```

For per-machine vars:

```
ls -la vars/per-machine/my-machine/my-generator/
```

3. Test Generators Locally

Create a test script to debug:

```
1 # test-generator.nix
2 { pkgs ? import <nixpkgs> {} }:
3 pkgs.stdenv.mkDerivation {
4   name = "test-generator";
5   buildInputs = [ pkgs.openssl ]; # Your runtime inputs
6   buildCommand = ''
7     # Your generator script here
8     mkdir -p $out
9     openssl rand -hex 32 > $out/secret
10    ls -la $out/
11  '';
12 }
```

Run with:

```
nix-build test-generator.nix
```

4. Enable Debug Logging

Set debug mode:

```
clan --debug vars generate my-machine
```

5. Check File Permissions

Verify generated secret permissions:

```
# On the target machine  
ls -la /run/secrets/
```

Recovery Procedures

Regenerate All Vars

If vars are corrupted or need refresh:

```
1 # Regenerate all for a machine  
2 clan vars generate my-machine --regenerate  
3  
4 # Regenerate specific generator  
5 clan vars generate my-machine --generator my-generator --regenerate
```

Manual Secret Injection

For recovery or testing:

```
# Set a var manually (bypass generator)  
echo "temporary-secret" | clan vars set my-machine my-generator/my-file
```

Restore from Backup

Vars are stored in the repository:

```
1 # Restore previous version  
2 git checkout HEAD~1 -- vars/
```

```
3  
4 # Check and regenerate if needed  
5 clan vars list my-machine
```

Storage Backend Issues

SOPS Decryption Fails

Symptom: "Failed to decrypt" or permission errors

Solution: Ensure your user/machine has the correct age keys configured. Clan manages encryption keys automatically based on the configured users and machines in your flake.

Check that:

1. Your machine is properly configured in the flake
2. Your user has access to the machine's secrets
3. The age key is available in the expected location

Password Store Issues

Symptom: "pass: store not initialized"

Solution: Initialize password store:

```
export PASSWORD_STORE_DIR=/path/to/clan/vars  
pass init your-gpg-key
```

Getting Help

If these solutions don't resolve your issue:

1. Check the [clan-core issue tracker](#)

2. Ask in the Clan community channels

3. Provide:

- The generator configuration
- The exact error message
- Output of `clan --debug vars generate`

← Previous

Advanced Vars Examples

Next →

Using Age Plugins with Clan Vars

Need help? Reach out to [our community](#).